What the course is about:

1. Theme: Techniques for designing efficient (scalable) algorithms
Inherent complexity/hardness of problems.

1a. Pre-requisite: CS 130A or equivalent. Mastery of
Algorithm Analysis framework (Big-Oh, worst-case)
Data Structures (hashing, heaps, BSTs, graphs)
How to present algorithms (describe, proof of correctness, analysis)

1b. Introduction
Algorithms and algorithmic thinking are pervasive.
"Algorithms" may well be the most popular tech-term in popular press today!

Google search algorithm
Facebook newsfeed and personalization algorithms
Targeted marketing algorithms
Algorithmic stock market trading
Algorithmic medicine

20-30 years ago, algorithms might have been a purview of core CS teaching.

Internet changed all that... It ``democratized'' computing, and today the reach of algorithms extends to nearly every aspect of scientific inquiry, not to mention
Commerce (amazon, ebay),
Education (MOOCs, online),
Entertainment (streaming videos, Pixar),
Finance (algorithmic trading, agent-based auctions)
Linguistics (automated translation)
Cloud Computing and Data Centers
Monitoring and Tracking
Robotics, etc etc..

Computational Biology: breakthroughs in genetic decoding (Human Genome Project), synthetic drug design etc

Economics/Finance: Computational thinking has become important to economists as mechanisms like auctions are being adopted for allocation of resources such as wireless spectrum, goods and services, key word ads on web pages etc.

In fact, much of CS can be thought of at its core a problem of "resource allocation"---computation, memory, communication, cache etc. As systems grow to global scale, these interactions sound a lot like an "economy" and therefore many classical ideas from economics are finding a key role in CS:
game theory, mechanism design etc.
Think of Cloud Computing and Data Centers.

Online Social Networks, such as Facebook and Twitter, have revolutionized the way social science research can be performed at massive scale...

Whether we want our web connections to work seamlessly, or the search engines to find the most relevant documents, or participate in virtual social groups, or play massive online games, the speed and scale at which information must be exchanged, managed, and processed is a nothing short of a miracle, and we often forget those silent algorithms working behind the internet cloud that make this possible.

1c. Algorithmic Enterprise

Algorithmic problems form the core of computer science, but they *rarely* arrive as cleanly packaged, mathematically precise questions. Rather real-world problems almost always come bundled with a lot of messy, confusing, application-specific detail, some of which essential, much of it extraneous and distracting that only clouds the mind.

As a result, the algorithmic enterprise consists of two fundamental, and equally important, parts:
o. extracting the mathematically clean core of the problem, and
o. designing an appropriate algorithm to solve the core problem.

The skills needed are interrelated: the more comfortable one is with algorithmic design techniques, the easier it is to recognize the clean abstraction and formulations that lie within the messy problems out in the world; and the more adept one is at sniffing out the right mathematical formulation, the easier it is often to make progress on the algorithm design question.

1d. Course goals:

What is the best algorithm for a given problem.
Three things you will learn:
Design a good algorithm, Analyze it, Know when to stop (lower bounds).

The word ``algorithm'' derived from Mohammed Al-Khowarizmi, 9th century Persian mathematician. He formalized pencil-paper methods for arithmetic.

(So, the abstract idea of an "algorithm" vastly predates computing technology, and the notions of complexity, Computability, efficiency are as fundamental as pure mathematics.)

Algorithms can be thought of as recipes---detailed and precise, where elegance, efficiency, simplicity etc matter.

2. Some Famous Algorithms
Constructions of Euclid, Newton's root finding, Fast Fourier Transform,
Data Compression (Huffman}, Lempel-Ziv, GIF, MPEG),
Data encryption (DES, RSA),
Simplex algorithm for linear programming,
Shortest Path Algorithms (Dijkstra}, Bellman-Ford),
Dynamic programming,
Error correcting codes (CDs, DVDs),
TCP congestion control, IP routing,
Pattern matching (Genomics),
Delaunay Triangulation (FEM, Simulation),

3. Topics Covered:
Models, Asymptotic notation, Worst-case analysis of algorithms,
Recurrences,
Greedy paradigm (Activity Selection, Huffman Coding, Shortest
Paths (Dijkstra), Min Spanning Trees (Kruskal, Prim))

Divide and conquer (Multiplying Long Numbers, Matrix Multiplication,
Quicksort Algorithm, Selection, Convex Hulls).

Dynamic Programming (Matrix Chain Product, Longest Common Subsequence,
Knapsack).

NP-Completeness.
Approximation Algorithms
Lower Bounds.

```
4. Why bother about investing in algorithms?

        Moore's Law: Computers are so fast, why bother with algorithms.

        Remember the Max Subsequence Problem from CS 130a.
                O(n^3), O(n^2), O(n log n) and O(n) algorithms.

        Even for a modest value n = 10^6, the first two algorithms are impractical.

        For more complex problem, the situation is much worse.

        TSP: Shortest route to visit N cities. An important optim. problem.

                For N = 100, we have 100! = 30^{100} possible tours
                A supercomputer checking 100 billion tours per sec
                        only does 10^{20} tours in 1 year!
                Still needs > 10^{100} years.


        Factoring:
                Fast number factoring algorithms can break encryption schemes.
                Straightforward method need millions of years to factor a
                        300-digit number.
                State of the art algorithm can break 100-digit numbers
                        in few days using network-of-workstations.
                Algorithms research determines what's considered safe
                        code lengths.

        Finally, Moore's Law is being drowned by much more rapid growth of data...
        More and more applications need to rely on sub-linear algorithm.

5. Tables of running times.

                        TABLES.

        Asymptotic estimates are good enough to make a high level
        distinction between practical and impractical algorithms.

        More refined analysis and/or simulations are used to make
        these judgments among multiple competing "theoretically
        equivalent" algorithms.


6. Review Asymptotic Notation; basic summations, and elementary
        recurrences. (Chapter 2) We will do more systematic
        study of recurrences later in the course.



7. Do divide-and-conquer algorithm for two-dimensional closest pair problem.
```